Runtime Enforcement of Safe Lane-Change Behavior in Autonomous Vehicles

Reading Elective Report

Nirbhay Sharma Harsh Gupta

Supervisor: Prof. Saumya Shankar

December 8, 2025

Abstract

This report summarizes our reading-elective work on runtime verification, automata learning, and a practical project: *runtime enforcement of safe lane-change behavior in autonomous vehicles* using DFA-based automata learning. This semester's work focused on studying runtime monitoring techniques, the L* algorithm and oracles for automata learning, and early simulation experiments in CARLA to prototype a two-vehicle lane-change scenario. The present document includes Introduction, Problem Statement/Motivation, and Literature Survey. Implementation and evaluation are ongoing.

1 Introduction

Autonomous Vehicles (AVs) increasingly rely on machine-learned controllers to make critical driving decisions such as lane keeping, following distance, and lane-changing. Although powerful, these controllers face several challenges:

- **Perception Noise:** Sensor inaccuracies may misinterpret nearby vehicles or road markings.
- **Planning Errors:** The decision model may incorrectly estimate safe maneuver timing.
- **Environmental Uncertainty:** Other road users behave unpredictably (e.g., sudden cut-ins).

These factors can lead to command sequences that violate expected safety rules, especially in high-risk maneuvers like lane changes. Offline techniques (e.g., static verification, model checking) are important but cannot handle all real-world execution conditions.

—

Runtime Verification (RV)

Runtime Verification ensures safety *during* execution by monitoring system eventsand checking them against a formal specification of allowed behavior. If a violation occurs, the system can block the unsafe action or enforce a safer alternative. Key RV ideas studied in this course:

- Event-based trace monitoring
- Online vs. Offline analysis
- Monitors synthesized from formal models such as DFAs or Temporal Logic

.

—

Automata Learning

Constructing a correct formal specification manually is error-prone. Automata Learning helps solve this by automatically inferring a deterministic finite automaton (DFA) from examples of safe behavior.

We studied:

- Angluin's L* active learning algorithm
- Use of Membership + Equivalence oracles (RandomWalk, W-method, etc.)
- AALpy library for practical learning experiments

.

—

Our Project Scope

We selected the domain of **Safe Lane-Change Enforcement** in AVs:

- Extract event traces from CARLA (e.g., `signal_on`, `gap_ok`, `lane_change`)
- Learn a DFA representing only **safe** lane-change sequences
- Use this DFA as a **runtime enforcer** (shield)

.

For this semester's prototype, we:

- Generated random traffic using CARLA utilities
- Implemented waypoint-navigation scripting

- Simulated a **forced two-vehicle overtake** scenario — Rear overtakes Front using lane-change motion

.

—

Goal of the Work

- Learn safe maneuver rules automatically
- Monitor and enforce compliance at runtime
- Prevent unsafe lane-change action sequences

This introduction reflects the core theoretical study and early experimental validation performed during this semester. Full evaluation and generalization across broader driving contexts will continue as future work.

2 Problem Statement and Motivation

Lane-change is one of the most safety-critical maneuvers in autonomous driving. Unlike lane keeping, it requires multiple coordinated checks and decisions:

- Identify and activate the correct turn signal
- Ensure safe longitudinal and lateral distance to nearby vehicles
- Execute a smooth and collision-free lane transition

However, AV decision-making pipelines are often dependent on machine-learned controllers and heuristic cost functions that do not guarantee safety in every real scenario. This leads to three key risks:

1. **Perception Faults:** Sensor noise or occlusions may incorrectly detect gaps.
2. **Planner Mistakes:** Unsafe maneuvers may be initiated without proper preconditions (e.g., missing signal state or insufficient gap).
3. **Unpredictable Traffic:** Other vehicles may accelerate, brake, or change lanes suddenly, invalidating the initial decision.

Safety standards in intelligent transportation systems require:

- **No safety violations** even under uncertainty,
- **Real-time response** to evolving traffic scenes.

Offline verification and testing cannot guarantee these requirements alone. Thus, we need a mechanism that:

- continuously monitors execution traces,
- blocks or corrects unsafe decisions before they propagate to control,
- ensures that maneuver steps strictly follow a verified safe sequence.

Motivation for Using Automata Learning

Handcrafting state-machine specifications for AV maneuvers is complex and prone to oversights. Automata learning enables:

- automatic extraction of **safe event sequences** from example traces,
- a **formally checkable model** of behavior (DFA),
- adaptability as more data is collected.

.

Problem Statement

How can we ensure that every autonomous lane-change operation follows a verified safe ordering of actions at runtime, even when the underlying controller may produce unsafe decisions?

To address this, we propose:

- Learning a DFA that models safe lane-change behaviors,
- Using the DFA as a runtime enforcer (shield),
- Demonstrating feasibility through a simulated two-vehicle overtake case in CARLA.

.

3 Literature Survey / Related Work

This section reviews the foundational concepts and research that support our work in runtime enforcement of autonomous vehicle lane-change behavior. The survey spans three core domains: (i) runtime verification and enforcement techniques, (ii) automata learning—especially DFA inference using active learning, and (iii) prior studies applying learning-based safety models in autonomous driving contexts.

Runtime Verification & Enforcement

Runtime Verification (RV) focuses on monitoring actual execution traces to detect and mitigate incorrect behaviors. Unlike offline verification methods such as formal model checking or theorem proving, RV is lightweight and deployable within real systems where specifications may be incomplete or the environment unpredictable.

Key concepts studied in the reading elective include:

- Trace monitoring of events and state transitions,
- Monitor synthesis from formal specifications such as DFAs or LTL,
- Online intervention for safety-critical decisions,
- Trade-offs between monitoring timeliness vs. system overhead.

Runtime *enforcement* (also called shielding) extends RV by not only detecting but also **preventing** unsafe actions through one of the following modes:

1. **Blocking Mode:** Unsafe actions are vetoed.
2. **Correction Mode:** The enforcer substitutes the nearest safe alternative.
3. **Recovery Mode:** System is guided back into safe states.

Prior influential works have applied shields to cyber-physical and embedded systems where correctness violations can cause physical harm:

- M. Abadi et al. — RV for network security policy enforcement
- Pnueli et al. — runtime monitors for reactive systems
- Bloem et al. — Shield synthesis for safety properties in real-time controllers

Such methods inspire our AV safety enforcer design for lane-changing.

Automata Learning for Safety Specifications

Finite automata are widely used to represent safety rules in a mathematically precise way. However, building these manually for complex systems is difficult. Automata learning automates this process by inferring a minimal DFA from examples:

$$\text{Data (traces)} \rightarrow \text{DFA model of allowed sequences} \tag{1}$$

We studied active learning techniques where the learner queries a system-under-learning (SUL) to classify traces:

- Angluin's L* algorithm — learns minimal DFAs using membership and equivalence queries.
- Practical query mechanisms — Random-walk, W-method, and StatePrefix oracles.
- AALpy framework — rapidly experiments with learned DFAs on real systems.

Strengths of automata learning for safety:

- Does not require a full formal model of the system,
- Can incrementally refine safety rules as more safe traces are collected,
- Supports mathematical reasoning and formal verification over learned DFAs.

Recent works applying L* or RPNI demonstrate safety rule inference in:

- Network protocol compliance testing,
- Hardware component verification,
- CPS / robotics behavior modeling.

These results justify using learned automata in AV runtime monitoring.

Safety in Autonomous Lane-Change

Lane changing encounters multiple interactive agents (vehicles, pedestrians), making it one of the most frequently cited sources of traffic accidents. Missteps include:

- poor gap estimation,
- failure to signal intentions,
- unsafe lateral acceleration during merge.

Recent literature explores combining formal safety layers with learning-based drivers:

- **Chen et al.** — Runtime monitors for highway merging constraints in AVs.
- **Alshiekh et al.** — Shielding deep RL policies to ensure safety constraints.
- **Camacho et al.** — Model predictive control with invariant safety sets.

Our approach is closely aligned with works enforcing *sequence-level* safety:

- Check turn signal on before change,
- Verify safe gap before lateral movement,
- Ensure stable driving after lane merge.

By extracting safe behavioral traces from CARLA simulations, we aim to learn a DFA that rejects unsafe lane-change event orders at runtime.

Summary of Research Gap

Most current AV safety systems focus on:

- **Spatial** safety constraints (collision avoidance, distance),
- **Control** safety (bounded acceleration and jerk),

but overlook:

- **Temporal and logical safety ordering:** e.g., lane-change must only occur *after* signaling and gap validation.

Automata learning + runtime enforcement directly addresses this gap by ensuring that:

Only verified safe behavioral sequences are ever executed by the AV controller.

This motivates our prototype system for enforcing safe lane-change behavior during execution.

4 Proposed Solution

Our proposed solution integrates concepts from runtime verification, automata learning, and autonomous driving simulation into a unified workflow for enforcing safe lane-change behavior. Since the present work focuses on building the foundational infrastructure, the solution described here is intentionally high-level. The concrete alphabet of events, the exact trace formats, and the final enforcement mechanism will be defined after completing data extraction and learning experiments.

Overview

The core idea is to construct a *learned safety automaton*—a DFA that represents only safe lane-change sequences—and to use this DFA as a runtime enforcer that prevents unsafe actions from being issued by an autonomous vehicle controller. The complete workflow consists of three main components:

1. **Simulation-Based Data Collection (CARLA)** We create controlled driving scenarios inside CARLA to generate maneuver trajectories. These scenarios include background traffic, navigation waypoints, and deterministic two-vehicle overtaking behavior. The purpose of this module is to produce execution traces containing the relevant variables and signals needed to characterize lane-change behavior. At this stage, the traces are generic time-indexed records; the symbolic alphabet used for learning will be derived later.

2. **Automata Learning (AALpy)** Once traces have been extracted from CARLA, they will be transformed into a symbolic sequence format suitable for DFA inference. We will then apply automata learning techniques—initially passive learners such as RPNI, with optional active refinement using L* oracles—to construct a minimal DFA that captures the structure of safe lane-change maneuvers. In this semester, we implemented two oracles from AALpy and validated their integration with a mock System Under Learning (SUL), forming the basis for future learning from CARLA-generated traces.

3. **Runtime Enforcement (Shielding)** The learned DFA will eventually serve as a *runtime shield* that intercepts controller outputs. If the controller attempts to issue an action that would lead the current execution trace into an unsafe or undefined DFA state, the shield will block or correct the action. In this report, we do not yet implement this enforcer; instead, we define the conceptual role it will play in the final system.

5 Implementation

The implementation carried out during this semester focuses on building the infrastructure required for (i) generating controlled driving scenarios in CARLA and (ii) testing automata-learning oracles using AALpy. At this stage, we do not yet extract symbolic traces or define the final event alphabet; instead, the work aims to ensure that (a) CARLA can reliably generate repeatable lane-change scenarios, and (b) AALpy oracles operate correctly on a mock System Under Learning (SUL). The full learning pipeline will be completed in the next phase.

This section summarizes the five implemented components.

1. Traffic Generation Module (generate_traffic.py)

The first requirement for realistic trace collection is a populated driving environment. The script `generate_traffic.py` is adapted from CARLA's official examples and configured to spawn vehicles and pedestrians using batch commands. It interacts with the Traffic Manager to handle:

- random vehicle placement,
- autopilot assignment,
- synchronous-mode execution for deterministic stepping,
- optional walker spawning with AI navigation.

This module provides the **background traffic context** in which our lane-change experiments will run. After integration with the enforcer in later work, this traffic will produce realistic disturbances and occlusions, ensuring that safety-rule learning and enforcement generalize beyond isolated scenarios.

2. Waypoint Navigation Module (waypoint_navigation.py)

The script `waypoint_navigation.py` establishes the basic navigation pipeline for autonomous driving inside CARLA. Its primary responsibilities are:

- configuring the CARLA Python client and map utilities,
- spawning a single vehicle and enabling autopilot,
- using CARLA's `GlobalRoutePlanner` to compute a sequence of navigation waypoints from a source to a destination,
- visualizing waypoints to verify route correctness.

The purpose of this module is to guarantee that vehicles possess a reproducible baseline driving trajectory. Lane changes in future learning experiments will be deviations from routes defined using this planner.

3. Deterministic Lane-Change Simulation (forced_overtake.py)

The script `forced_overtake.py` produces a controlled, reproducible two-vehicle lane-change maneuver. The setup consists of:

- a front vehicle moving steadily in the reference lane,
- a rear vehicle placed several meters behind,
- a forced right-lane overtake executed via transform interpolation.

Unlike PID steering or autopilot-based lane changes, our use of **interpolated transforms** ensures deterministic motion: every execution results in the same lateral and longitudinal trajectory. This predictability is crucial for:

- generating clean "safe" and "unsafe" maneuver templates,
- controlling key parameters (gap size, timing, velocity),
- debugging trace-collection procedures.

This script serves as the initial "System Under Learning" from which symbolic traces will eventually be extracted.

4. W-Method Equivalence Oracle (wmethod.py)

To prepare the automata-learning pipeline, we implemented and evaluated AALpy's W-Method equivalence oracle in `wmethod.py`:contentReference[oaicite:3]index=3. Since CARLA traces are not ready yet, we used a mock SUL (an even-parity DFA) to validate:

- correct operation of the L* learning loop,
- stability and correctness of membership queries,
- systematic coverage of the hypothesis automaton's state space.

The code successfully learns a minimal DFA using systematic test generation, confirming that the oracle will work once CARLA-derived traces are integrated.

5. Random Walk Equivalence Oracle (randomwalkoracle.py)

The script `randomwalkoracle.py`:contentReference[oaicite:4]index=4 implements the RandomWalk equivalence oracle from AALpy. This oracle:

- performs long random explorations of the SUL,
- probabilistically resets the SUL to discover rare behaviors,
- exposes inconsistencies between the learned DFA and the true system.

As with the W-Method oracle, we validated the implementation using the even-parity SUL. The success of both oracles ensures that our learning infrastructure is ready for integration with CARLA-generated traces.

Summary of Implementation

The five components implemented this semester establish a working foundation for:

1. generating dynamic and deterministic CARLA scenarios,
2. navigating vehicles using waypoint sequences,
3. producing controlled lane changes suitable for trace extraction,
4. running automata learning with both systematic and random exploration.

With these modules complete, the system is prepared for the next stage: extracting symbolic event traces from CARLA and training a DFA that models safe lane-change behavior.

6 Future Scope

The next phase of this project focuses on linking the CARLA-driven simulation infrastructure with the automata-learning pipeline. While this semester established the foundational components, several key developments remain:

- **Event Extraction and Alphabet Design:** Defining a symbolic alphabet based on CARLA telemetry such as vehicle state, turn-signal activation, relative gap measurements, and lane-change progression.
- **Trace Collection and Labeling:** Recording both safe and unsafe maneuver sequences from the deterministic two-vehicle overtake scenario and from runs involving background traffic. These traces will serve as input for passive and active learning.
- **Learning a Safety Automaton:** Applying AALpy's RPNI (passive) and L* (active) algorithms to infer a DFA that captures the temporal constraints of safe lane changes, and refining the learned model using equivalence oracles.
- **Runtime Enforcer Integration:** Transforming the learned DFA into a runtime shield that intercepts controller outputs and blocks transitions that violate the safety specification.
- **Generalization and Evaluation:** Extending experiments to multi-lane, multi-agent, and higher-speed settings, and evaluating robustness under perception uncertainty and adversarial conditions.

These tasks will complete the end-to-end pipeline from simulated behavior to a formally grounded runtime safety mechanism.

7 Conclusion

This semester's work successfully established the theoretical and practical foundation for runtime enforcement of safe lane-change behavior in autonomous vehicles. On the theoretical side, we studied runtime verification, monitoring, and active automata learning, including Angluin's L* algorithm and the role of membership and equivalence oracles.

On the practical side, we built the necessary CARLA simulation infrastructure: traffic generation, waypoint navigation, and a deterministic two-vehicle lane-change scenario designed for reproducible trace extraction. We also implemented and validated two AALpy equivalence oracles—the W-Method and Random Walk—using a mock System Under Learning, ensuring that the automata-learning pipeline is ready for integration with real driving traces.

While the final learned safety automaton and runtime enforcer are yet to be constructed, the groundwork laid this semester provides all essential components for completing the

learning and enforcement stages. This project demonstrates the feasibility of combining simulation-driven data, automata learning, and runtime verification to enhance safety in autonomous driving systems.

Future work will focus on extracting symbolic traces, learning a safety DFA, and deploying it as a runtime shield capable of preventing unsafe lane-change sequences during execution.

References

- **Project GitHub Repository.** N. Sharma, H. Gupta. *Runtime Enforcement of Safe Lane-Change Behaviour in Autonomous Vehicles.* Available at: `https://github.com/NirbhaySharma504/Runtime-Verification`

- **AALpy: Active Automata Learning Library.** AALpy GitHub Repository. Available at: `https://github.com/DES-Lab/AALpy`

- **CARLA Simulator.** CARLA Autonomous Driving Research Simulator. Available at: `https://github.com/carla-simulator/carla`

- **L* Algorithm Original Paper.** Dana Angluin, "Learning Regular Sets from Queries and Counterexamples," *Information and Computation*, 75(2), 1987. Available at: `https://doi.org/10.1016/0890-5401(87)90052-6`

- **Lectures on Runtime Verification.** Ezio Bartocci and Yliès Falcone (Eds.). *Lecture Notes in Computer Science, Vol. 10457.* Springer, 2017. Available at: `https://link.springer.com/book/10.1007/978-3-319-66201-1`